ID3802 – Monthly Progress Report

Project ID : P191

Project ID	P191
Date	29-01-2026
Project Title	Necessarily Rank-Maximal Matching
Students	Madhav P Nair (112301020), Devapriya Pradeep (112301006)
Mentor	Dr.Pratik Ghosal

1 Work done in the current month

1.1 Project overview

The Necessarily Rank-Maximal matching is a niche but promising area in graph theory. Here we address the problem of resource allocation and try to optimize it using an optimality criterion called Rank maximality. We deal with the cases where agents have preferences over some objects but not vice versa.

Consider a set of students and a set of projects where each student has a preference list in which each project gets a specific rank. We work with the online variant of the problem where we start with an empty preference table and elicit the preferences of the students over the projects using a **query model** and return an optimal matching once sufficient information is retrieved without any further queries.

1.2 Current Progress

Jannik Peters[1] put forward an open question in his paper to improve the upper bound of the online elicitation algorithm for rank maximal matching based on the set compare model which is $O(n)$(trivial) where n is the number of agents/objects.We addressed this open challenge and modified the set compare model without deviating from its basic principles, but retrieving more information from an agent in each query. We propose the **Rank-Aware set-compare model** based on this. We came up with an **original algorithm** for the online elicitation of rank maximal matching based on the RASC model.

1.3 Approach

An optimal algorithm is a hypothetical algorithm that has complete knowledge of the entire input in advance and uses this knowledge to determine the minimum number of queries needed to compute an NRM matching.
We say that an online elicitation algorithm is α-competitive or has competitive ratio α if the number of queries it requires to ask in the worst-case across all possible instances is at most $\alpha \cdot \text{OPT}$ where OPT is the number of queries asked by the optimal algorithm.
We focus on the analysis and proof of the algorithm presented. We prove (or disprove) that the algorithm has a constant competitive ratio, which is an improvement from the linear competitive ratio of the set compare query model.

2 Feedback from the mentor

The progress of the students is satisfactory.

3 Signature of the mentor

Pratik Ghosal
31/01/2026

References

[1] Jannik Peters. Online elicitation of necessarily optimal matchings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 5164–5172, 2022.

ID3802 – Monthly Progress Report (February)

1 Work done in the current month

1.1 Literature Survey

We read the work of Peters on Online Elicitation of Optimal Matchings [1].Our original algorithm is essentially inspired from the open question proposed by Peters in his paper on Online Elicitation of Necessarily Optimal Matchings. It is very fundamental to comprehend his algorithms, the corresponding optimal offline algorithms and the techniques he employed to come out with competitive ratios(α) for moving forward. He gives algorithms for online elicitation of necessarily rank-maximal matchings based on both next-best query model and hybrid query model. Our primary focus was on the online algorithm he proposed based on the next-best query model, the proof of correctness, and competitive ratio analysis for the same.

The algorithm is followed by the correctness proof of the algorithm. The author introduces a couple of terms and related constraints which are helpful in the upcoming α - analysis. After setting up the plot, the author goes for deriving a upper bound of 3 for α. This is followed by the rigorous analysis for the tight upper bound of 3/2.

1.2 Plot

Lemma 1.1. *Let $rev(A)$ denotes the set of agents who are matched to a revealed post. Then in any instance of necessarily rank-maximal matching problem, every $a \in rev(A)$ must have at least $r_a - 1$ revealed posts, where r_a is the first iteration where a becomes an odd or unreachable agent for the first time.*

That is, atleast $|r_a - 1|$ queries are needed for each agent a by the optimal algorithm to determine the NRM.

1.3 Upper bound

The author adopts a classic strategy in competitive analysis. This approach starts by kind of glorifying the OPT to create a safety margin (constant-competitive) and later applies the real structural constraints and thus grounds the adversary.
It starts with some assumptions for this loose analysis

- The optimal algorithm should match at least $n - 1$ agents to revealed posts
- If an agent is matched to a revealed post, $r_a - 1$ preferences must be revealed for this particular agent
- In our algorithm, each agent has to reveal r_a posts

So,

$$ALG \leq \Sigma(r_a) + n - 1$$

and

$$OPT \geq \Sigma_{a \in A'}(max(r_a - 1, 1))$$

Thus we get

$$\frac{ALG}{OPT} \leq \frac{\sum_{a \in A'}(r_a) + n - 1}{\sum_{a \in A'}(\max(r_a - 1, 1))} \leq \frac{\sum_{a \in A'}(r_a + 1)}{\sum_{a \in A'}(\max(r_a - 1, 1))}$$

$$\frac{\sum_{a \in A'}(r_a + 1)}{\sum_{a \in A'}(\max(r_a - 1, 1))} \leq \max_{a \in A'} \left(\frac{r_a + 1}{\max(r_a - 1, 1)} \right) \leq 3$$

1.4 Tight analysis for 3/2

Here we employ more rigorous structural comparisons and derive the tight bound of 3/2 for α. Instead of isolating agents, the method accounts for **resource conflicts** by **grouping agents** who compete for the same house which forces the OPT also to work harder to resolve conflicts.

1.4.1 The unrevealed case

Suppose agent a' matches to an unrevealed house. Then this leftover house should be the last preference for all other agents or none of the houses in the revealed matchings are revealed by a'. Otherwise they could swap houses to make a better matching.

Here, OPT has to query every preference of each agent who match to a revealed house to make sure that the leftover house is their worst preference and ALG seeks every preference of all agents.
OPT $= (n-1)^2$
ALG $= n(n-1)$

$$\frac{ALG}{OPT} = \frac{n}{n-1}$$

which is at most 3/2 for $n \geq 2$

1.4.2 Grouping by first choice

The author groups agents by the first-choice house to capture the resource contention. A_h is defined as the set of all agents whose first choice is house h. OPT should ask at least $2|A_h|$-1 queries to this group which can be interpreted as two queries each to all the losers who have better fallback option as the second choice and one query to the winner. $OPT(A_h)$ is the minimum number of queries the optimal algorithm must ask the specific group of agents who rank house h as their first choice.

$$OPT(A_h) \geq max(\Sigma_{a \in A_h}(max(r_a - 1, 1), 2|A_h| - 1)$$

$$= \max(\text{individual necessity}, \text{group conflict})$$

$$OPT = \Sigma_{h \in H} OPT(A_h)$$

$$ALG = \Sigma_{h \in H} \Sigma_{a \in A} r_a$$

The author applies the powerful **mediant rule** ie.,

$$\frac{\Sigma a_i}{\Sigma b_i} \leq max(\frac{a_i}{b_i})$$

Thus the upper bound reduces to

$$\frac{OPT}{ALG} \leq max_{h \in H} \frac{\Sigma_{a \in A_h} r_a}{OPT(A_h)}$$

Now we have two cases :

1. **High query case:** $\sum_{a \in A_h} r_a \geq 3|A_h|$

$$\frac{OPT}{ALG} \leq \max_{h \in H} \frac{\sum_{a \in A_h} r_a}{OPT(A_h)} \leq \max_{h \in H} \frac{\sum_{a \in A_h} r_a}{\max\left(\sum_{a \in A_h} \max(r_a - 1, 1),\ 2|A_h| - 1\right)} \leq \max_{h \in H} \frac{\sum_{a \in A_h} r_a}{\sum_{a \in A_h} \max(r_a - 1, 1)}$$

$$\frac{OPT}{ALG} \leq \max_{h \in H} \frac{\sum_{a \in A_h} r_a}{r_a - |A_h|} \leq \frac{\sum_{a \in A_h} r_a}{r_a - \frac{1}{3}\sum_{a \in A_h} r_a} = \frac{\sum_{a \in A_h} r_a}{\frac{2}{3}\sum_{a \in A_h} r_a} = \frac{3}{2}.$$

2. **Low query case:** $\sum_{a \in A_h} r_a < 3|A_h|$
   There exists at least one $a \in A_h$ with $r_a = 2$ since $\sum_{a \in A_h} r_a < 3|A_h|$.
   Let $A_h^2 = \{a \in A_h \mid r_a = 2\}$.
   (a) $|A_h^2| \geq 2$

$$\frac{\sum_{a \in A_h} r_a}{\mathrm{OPT}(A_h)} = \frac{\sum_{a \in A_h \setminus A_h^2} r_a + 2|A_h^2|}{\mathrm{OPT}(A_h \setminus A_h^2) + \mathrm{OPT}(A_h^2)}$$

$$\leq \max\left(\frac{\sum_{a \in A_h \setminus A_h^2} r_a}{\sum_{a \in A_h \setminus A_h^2} r_a - |A_h \setminus A_h^2|}, \frac{2|A_h^2|}{2|A_h^2| - 1}\right) \leq \frac{3}{2}.$$

Since $\sum_{a \in A_h \setminus A_h^2} r_a \geq 3|A_h \setminus A_h^2|$ and it had already been proven that for such a case, $\frac{ALG}{OPT} \leq \frac{3}{2}$, the first argument for max is bounded by $\frac{3}{2}$.

$$\frac{2|A_h^2|}{2|A_h^2| - 1} \leq \frac{3}{2} \quad \text{for all } A_h \text{ such that } |A_h| \geq 2.$$

Therefore, we get an upper bound of $\frac{3}{2}$.

(b) $|A_h^2| = 1$

There is exactly one $a \in A_h$ with $r_a = 2$. So a is O or U in the 2nd iteration. Since all other agents are E in the 2nd iteration and they all have edges to h, h can't be even in the second iteration since EE edges are not allowed. So (a, h) Since h is unavailable, and a gets out of E in the 2nd iteration, a must be matched to it's 2nd preference. So, OPT must ask a 2 queries.

$$\frac{ALG}{OPT} \leq \frac{\sum_{a \in A_h} r_a}{\max\left(\sum_{a \in A_h} \max(r_a - 1, 1),\ 2|A_h| - 1\right)} \leq \frac{3|A_h| - 1}{2|A_h| - 1} \approx \frac{3|A_h| - 1}{2|A_h|} = \frac{3 - \frac{1}{|A_h|}}{2} \leq \frac{3}{2}.$$

Thus, the algorithm has a competitive ratio of $\frac{3}{2}$.

2 Feedback from the mentor

Satisfactory.

3 Signature of the mentor

Pratik Chhosal
01/03/2026

References

[1] Jannik Peters. Online elicitation of necessarily optimal matchings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 5164–5172, 2022.

ID3802 – Monthly Progress Report (March)

1 Work done in the current month

1.1 Lemma 5 of Peters

Lemma 5 of [1] acts as a vital step to show a constant bound for the competitive ratio for the algorithm proposed by Jannik Peters. This ensures that the competitive-ratio will have a constant bound of 3 and leaves room for further improvement.

Lemma 1.1. *Given a set of agents A, objects O and a partial preference profile $\succ$ in the next-best query model and an NRM M for the problem, $|rev(a)| \geq r_a - 1$ for all agents a matched to a revealed preference profile by M. Here r_a is the iteration at which agent a is no longer even.*

We assume to the contrary that there exists an agent a with $M(a) \in rev(a)$ and $|rev(a)| < r_a - 1$. We know that $a \in \mathcal{E}_{r_a-1}$ by the definition of r_a, which means that there is an even length alternating path from a to a free vertex a_k. Since a_k is free, the pair matched with a_k in M must have a rank higher than $r_a - 1$, i.e., $rank(a_k, M(a_k)) > r_a - 1$.

Assume that there exists an a_i with $i > 0$ such that $rank(a_i, M(a_{i-1})) > rank(a_{i-1}, M(a_{i-1}))$ for a_i in the even alternating path between a and a_k. Since a is even, all a_i in the path are also even in the $(r_a - 1)$st iteration. This means that the pair matched to a_{i-1} becomes odd at the iteration at which it is matched, i.e., $M(a_{i-1}) \in \mathcal{O}_{rank(a_{i-1}, M(a_{i-1}))}$. Since the algorithm treats odd objects as inactive, they do not participate in any further path augmentations. Therefore, $M(a_{i-1})$ cannot be matched with an agent that is not a_i in G_{r_a-1}. That is, the edge $(a_i, M(a_{i-1})) \notin G_{r_a-1}$. This contradicts our assumption, and therefore, $rank(a_i, M(a_{i-1})) \leq rank(a_{i-1}, M(a_{i-1}))$ for all a_i in the alternating path with $i > 1$. Furthermore, $\succ$ can be extended so that $rank(a_1, M(a_k)) \leq r_a - 1$.

Taking into account the even length alternating path starting from a and ending at a_k, we can augment M to get a new matching M' which has $rank(a_i, M'(a_i)) = rank(a_i, M(a_{i-1})) \leq rank(a_{i-1}, M(a_{i-1}))$ for all a_i, $i > 1$ and $rank(a_1, M(a_k)) \leq r_a - 1$ while $rank(a_k, M(a_k)) > r_a - 1$ previously. Thus, M' is strictly a better rank-maximum matching compared to M. This contradicts our assumption that M is NRM and therefore $|rev(a)| \geq r_a - 1$ for all $a \in A$.

1.2 Similar arguments Observations for the RASC based algorithm

From the preliminary analysis on the performance of the RASC based algorithm, one can easily observe that the ALG is able to give a tough competition to the OPT since it gets the "**skip**" power like OPT thanks to the rank information as opposed to the vanilla set compare model.

Moreover, we are able to delete the Odd and Unreachable objects (permanently matched objects) from the agent - specific query subset H_i of agents due to the same reason because the edge corresponding to the next best choice of the agent from her available set of houses will only be added in the correct iteration. Thus, the size of the query set is drastically reduced, which directly influences the total number of queries asked. This property was also absent in the normal set-compare model.

What we observed in the preliminary analysis is that the argument put forward in lemma 5 can be extended to our algorithm also since the number of queries asked by the algorithm is r_a and the lemma forces the OPT to be at least $r_a - 1$. This gives us a constant bound of 3 for α.
These observations will be helpful for the later part of the analysis to derive a tighter bound.

1.3 Implementation

We have initialized an exhaustive **repository** for Necessarily Rank-Maximal matchings which documents the required background and our research work on RASC-model and the original algorithm based on it. We have also implemented the online algorithm in python in a sandbox environment using Networkx package.

Implementation details :

- The library (`/implementation/lib`) has two modules :
  - `graph-utils.py` for basic functions on bipartite graphs like construction, updation, computing maximum matching etc.

  - `edmonds-gallai.py` contains the function for computing Edmonds - Gallai decomposition by taking a graph and a maximum matching on it as input

- `/implementation/algo.py` uses the modules to implement the RASC-based algorithm for eliciting rank-maximal matchings. It queries user preferences in the run-time.

The correctness of the code has been tested using a variety of preference profiles as inputs. Additionally, the algorithm is now able to catch an agent if she gives **inconsistent/fake preferences** and raises execption. This is the first step to set up the backend engine and to develop the frontend interface of the simulation.

The tentative tech stack for implementing the simulator is as follows :

- FastAPI (Python) with Networkx for backend
- ReactJS for the frontend
- The MVP (Minimum Viable Product) plans to use in-memory dictionaries for both state management and data storage

The github repository link : Necessarily rank-maximal matchings repo

2 Feedback from the mentor Satisfactory

3 Signature of the mentor Pratik Ghosal

31/03/2026

References

[1] Jannik Peters. Online elicitation of necessarily optimal matchings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 5164–5172, 2022.

ID3802 – Monthly Progress Report (April)
Project ID : P191

1 Work done in the current month

1.1 Analysis : Lemma to bound OPT

Here, we try to show that the OPT has to do at least ALG - 2 queries to ensure that the matching produced by it will remain rank-maximal irrespective of how the unrevealed preferences are filled.

1.1.1 Lemma

In RASC-based NRM elicitation, the OPT has to perform at least ALG - 2 queries.

$$OPT \geq ALG - 2$$

We try to produce an instance where the OPT ends up asking 3 queries less than the ALG and claims it has got a rank-maximal matching. We do a case-by-case analysis of the above scenario, disprove the claim of OPT and thus prove the lemma by **contradiction**.

Consider an agent a. The following illustrates the preferences of a revealed by the RASC-based ALG.

Agent	$r=1$	$r=2$	$r=3$	$r=4$	$r=5$	$r=6$	$r=7$	$r=8$	$\cdots$	$r=k$
a	o_1	$\times$	$\times$	$\textcircled{o_4}$	$\times$	$\textcircled{o_6}$	o_7	$\textcircled{o_8}$	$\cdots$	o_k

o_i denotes the i^{th} choice of a and $r = i$ means i^{th} rank.
$k <= n$ (total number of agents or objects) and the agent a is Even till k^{th} iteration (since she is queried till k^{th} iteration).

Recall how ALG works. Some of the preferences of a are not queried by the ALG since the posts are already Odd or Unreachable by the time of that iteration and those skips are marked as **X**. Moreover, the ALG stores the next best choice of a from the available posts in memory in this iteration.

W.L.O.G, we say that in the partial preference list of a revealed by OPT, three choices - o_4, o_6 and o_8 are not revealed (marked in circles) and still OPT claims that it has produced a rank-maximal matching having (a, o), ie., agent a is matched to some object o.

We break down this choice of o into different cases and find a better matching in each case. We leverage the fact that we are free to **manipulate the positions which are not revealed by OPT** and can permute as we wish to prove OPT wrong.

We make use of the concept of **critical rank**[1]. Critical rank c of (a, p) is defined as the first iteration where either the agent a or the object o becomes Odd or Unreachable. For every $1 \leq ic$, the edge (a, p) belongs to every rank-maximal matching, in which (a, p) has rank i

The object o can be classified into five categories comparing its rank with the rank of unrevealed(OPT) objects. In each case, **we promote an object o^* unrevealed by OPT in the preference list to a position which ensures Even-ness for the object, so that $rank(a, p)$ is strictly less than its critical rank. This ensures (a, o^*) is matched in every rank-maximal matching.** We place the promoted object on or before the position where it was queried from. This object has to be Even (available) in this region.

1. $rank(a, o) < rank(a, o_4)$: swap the positions of o_4 and o_6. Every rank-maximal matching should contain (a, o_6) and so we discard the matching given by OPT.
2. $o = o_4$: swap the positions of o_4 and o_6. Every rank-maximal matching should contain (a, o_6). The matching given by OPT no more remains rank-maximal.
3. $o = o_6$: swap the positions of o_6 and o_8. We promote o_8 and match it with a. Every rank-maximal matching should contain (a, o_8) and thus we discard the matching given by OPT. **Here, it makes no sense to promote and use o_6 to match**

with a since $o6$ is the one given by OPT. This is the tighter case which forces us to have three unrevealed objects to efficiently manipulate the list.

4. $o = o_8$: swap the positions of o_4 and o_6. Every rank-maximal matching should contain (a, o_6) and thus we discard the matching given by OPT.

5. $rank(a, o) > rank(a, o_8)$: we can promote either o_6 or o_8 to get a better match for a which is obviously not o.

Here, in each case we could prove OPT is wrong. So, OPT cannot afford asking three or more queries less than the ALG and hence the Lemma 1.1.1 is proved.

1.2 Implementation : Backend for NRM simulator

- We implemented an asynchronous function that takes the number of agent-object pairs as input and returns a Necessarily Rank Maximal matching through an interactive process, by continuously querying users for their preference lists.
- A websocket endpoint '/ws/nrm' was developed which utilizes the function mentioned above and maintains a persistent communciation with the user.
- Apart from the implementation of the algorithm, various error handling mechanisms have also been added to spot inconsistencies in user input. They include:
  - Incomplete user input such as a missing object or object rank.
  - Inconsistent preferences such as input of a lower-ranked object after a higher ranked object or failing to provide a first choice in the first algorithm iteration.
  - Input of a preference that exceeds the number of agent-object pairs that exist.
  - Making a jump in the preference list by providing a less preferred object when a better one is available.
- A frontend for the application is currently under development.

2 Feedback from the mentor

Satisfactory.

3 Signature of the mentor 

References

[1] Pratik Ghosal and Katarzyna Paluch. Manipulation strategies for the rank maximal matching problem. October 2017.

Project ID : P191